

EE 446 - TinyML - Assignment 2

Total Points: 100

Summer Quarter, 2026

Department of Electrical and Computer Engineering

University of Washington, Seattle, WA

Due: 11:59 pm (PST) on August 12th (Wednesday), 2026 via Canvas

Note:

- This assignment contains both [programming questions](#) (marked as [\[Pro\]](#)), which require writing Python code, and [discussion questions](#) (marked as [\[Dis\]](#)).
- You may discuss concepts with your classmates; however, **all Python code and answers to discussion questions must be written independently**.
- Your assignment submission must be hosted in a [public GitHub repository](#). You should submit the [GitHub repository link](#) through Canvas as your assignment submission. Please do not submit the notebook and PDF files directly to Canvas.
- Your public GitHub repository must contain the following two files:
 - *i*) **Source code for all programming questions ([Pro])** as a [single Jupyter Notebook](#), named “[#.\\$_EE446_HW2_Pro.ipynb](#)”, where “#” and “\$” should be replaced with your first name and last name, respectively.
Clearly indicate which [\[Pro\]](#) problem each code block corresponds to, so your solution can be easily identified. Print the program output for each programming question. Include markdown cells and/or inline comments to improve readability.
 - *ii*) **Handwritten and scanned or typed answers** to all discussion questions ([\[Dis\]](#)) as a [single PDF file](#), named “[#.\\$_EE446_HW2_Dis.pdf](#)”, where “#” and “\$” should be replaced with your first name and last name, respectively.
- Your Canvas submission should contain only the public GitHub link to the repository containing the two required files above. Please make sure the link is accessible before submitting.

In this assignment, you will make use of the following dataset and files. You will find these files when you unzip the [EE446_HW2_Files.zip](#):

- **Network Anomaly Dataset**: This dataset consists of 125,973 data samples, each with 42 features corresponding to normal and attack network connections. The [network_anomaly_data.txt](#) file included with this homework assignment provides this dataset.
- **FirstName_LastName_EE446_HW2_Pro.ipynb**: You will use this Python notebook file to answer the programming questions in this assignment. When submitting your assignment, **please replace FirstName_LastName with your first and last name**.
- **c_writer.py**: This script will be called in [FirstName_LastName_EE446_HW2_Pro.ipynb](#) to automatically convert your TensorFlow Lite model to C and create the header file ([network_model.h](#)) for Arduino Nano 33 BLE Sense deployment.
- **network_data.ino**: You will use this Arduino sketch file for the deployment-related question in this assignment. You will find this file inside the [network_data](#) folder.

1. [Pro] (Data Preprocessing, 5 pts x 3 tasks = 15 pts)

Complete the following tasks in the provided Python notebook (.ipynb) file:

- (a) Drop the 'land', 'urgent', 'numfailedlogins', 'numoutboundcmds' columns from the dataframe *data*.
- (b) Change any label that is not named **normal** to **attack** in the {'attack'} column of the dataframe *data*.
- (c) Use **LabelEncoder()** function from the **sklearn.preprocessing** library to convert non-numerical attributes in the 'protocoltype', 'service', 'flag', 'attack' columns of the dataframe *data* to numerical values.

2. [Pro] (Dimensionality Reduction for Visualization, 10 pts x 3 tasks = 30 pts)

Complete the following tasks in the provided Python notebook (.ipynb) file:

- (a) Use **TSNE** from the **sklearn.manifold** library to visualize the data in the **test set (X_test)** in 2D. In your figure, use color **red** to mark **attack** data points and color **blue** to mark **normal** data points.
- (b) Use **PCA** from the **sklearn.decomposition** library to visualize the data in the **test set (X_test)** in 2D. In your figure, use color **red** to mark **attack** data points and color **blue** to mark **normal** data points.
- (c) Use **KernelPCA** from the **sklearn.decomposition** library to visualize the data in the **test set (X_test)** in 2D. Use **radial basis function (rbf)** as the kernel. In your figure, use color **red** to mark **attack** data points and color **blue** to mark **normal** data points.

3. [Pro] (Implementing a DNN on the dataset, 5 pts x 3 tasks = 15 pts)

Complete the following tasks in the provided Python notebook (.ipynb) file:

- (a) Implement a **deep neural network (DNN)** on the **Network Anomaly Dataset**. Ensure to include **one neuron** and **sigmoid activation** in the output layer of your DNN.
- (b) Compile and train your DNN model on the **training set (X_train)**. Denote the trained model as **base_model**.
- (c) Evaluate the **base_model** on the **test set (X_test)** using **classification_report** and **confusion_matrix** from the **sklearn.metrics** library. **Report these numbers in your .pdf writeup file using screenshots.**

4. [Pro] (Implementing a Full-Integer Quantized Model, 10 pts × 2 tasks = 20 pts)

Complete the following tasks in the provided Python notebook (.ipynb) file:

- (a) Implement full-integer INT8 post-training quantization on **base_model**. Use a representative dataset from **X_train** to calibrate the activation ranges. Configure the TensorFlow Lite model to use INT8 operators with INT8 input and output tensors. Designate the resulting quantized model as **tf_lite_quant_model**.
- (b) Evaluate **tf_lite_quant_model** on the test set (**X_test**) using **classification_report** and **confusion_matrix** from the **sklearn.metrics** library. Properly quantize each input sample using the input tensor's scale and zero point before inference. Report the results in your **.pdf** writeup file using screenshots.

5. **[Pro]** (Deploying the Quantized Model, 5 pts x 4 tasks = 20 pts)

Complete the following tasks in the provided Arduino sketch (.ino) file:

- (a) Implement code to obtain the **prediction** from the **output tensor** and determine the **predicted class label**.
- (b) Implement code to output **Sample #**, **Predicted Class**, and **Actual Class** for each sample to the serial monitor using **Serial.print** function.
- (c) Add the **network_model.h** file, generated at the end of the Python notebook, to the **network_data** folder where **network_data.ino** is kept. Upload the **network_data.ino** to the Arduino Nano BLE. **Obtain screenshots of the printed Serial Monitor outputs for five samples and report these in your .pdf writeup.**
- (d) Use the code at the end of the Python notebook to obtain **10 features and actual labels of the test set excluding the first five samples**. Add these new data at the appropriate places in the **network_data.ino**. Upload the **network_data.ino** to the Arduino Nano BLE. **Obtain screenshots of the printed Serial Monitor outputs for these ten samples and report these in your .pdf writeup.**